Lập trình mạng

Ống dẫn (Pipe) và Stream trong Node.js

TS.GVC. Nguyễn Văn Khanh
PLC & IIoT Lab

Mục tiêu và phạm vi chương

- Hiểu Stream/Pipe như một mô hình xử lý dữ liệu theo luồng trong Node.js.
- Chạy được các ví dụ: đọc file theo chunk, copy file, Transform Stream, nén file, xử lý log JSON.
- Biết kiểm tra kết quả ngay trên lớp bằng terminal và file đầu ra.
- Hướng tới lập trình mạng: chuẩn bị tư duy dữ liệu theo dòng, theo sự kiện và không block.

Chuẩn bị môi trường code

1. Cài Node.js và kiểm tra bằng lệnh: node -v.
2. Tạo thư mục thực hành: ctu-pipe-demo.
3. Mỗi ví dụ lưu thành một file .js riêng.
4. Chạy code bằng terminal: node ten_file.js.
5. Quan sát kết quả trên terminal và các file được tạo ra.

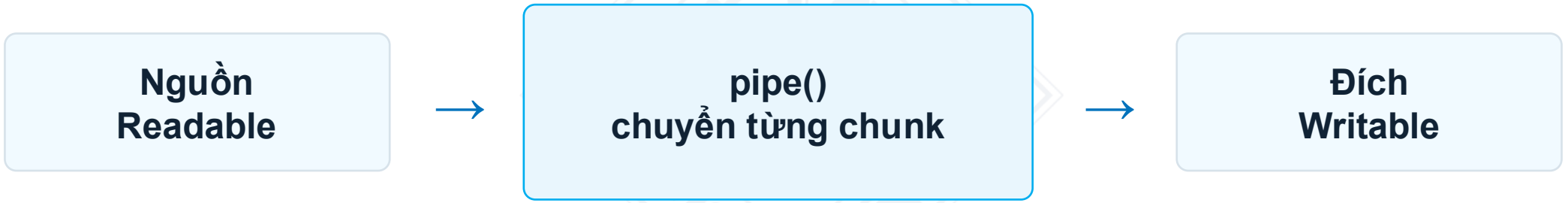
```
mkdir ctu-pipe-demo
cd ctu-pipe-demo
node -v

# chạy từng ví dụ
node 01-read-chunks.js
```

```
Kết quả kiểm tra:
v20.x.x
# hoặc phiên bản Node.js hiện có
```

Pipe là gì?

- Pipe cho phép dữ liệu đi tuần tự từ nguồn phát sang nơi nhận.
- Trong Node.js, pipe() thường dùng để nối các Stream.
- Dữ liệu không cần nạp toàn bộ vào RAM trước khi xử lý.




```
readable.pipe(writable);
```

4 loại Stream trong Node.js

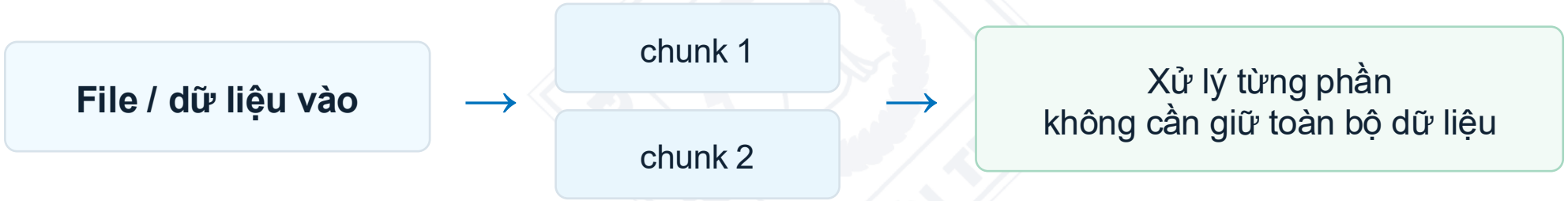
Readable	Writable	Duplex	Transform
Đọc dữ liệu từ nguồn	Ghi dữ liệu ra đích	Vừa đọc vừa ghi	Đọc → xử lý → ghi
createReadStream()	createWriteStream()	stdin/stdout	zlib, custom Transform

Mối liên hệ với lập trình mạng

Dữ liệu mạng cũng thường đến theo từng phần: gói tin, dòng log, bản tin cảm biến, JSON message. Stream giúp ta luyện tư duy xử lý dữ liệu không đồng bộ trước khi học Socket.IO.

Chunk và Buffer

- Stream không đưa toàn bộ dữ liệu một lần; dữ liệu đến theo từng chunk.
- Trong Node.js, chunk thường là Buffer.
- Buffer là vùng dữ liệu nhị phân; có thể chuyển sang chuỗi bằng toString().




```
stream.on('data', chunk => {
  console.log(chunk.length);
  console.log(chunk.toString());
});
```

Ví dụ 1: Đọc file theo từng chunk

Mục tiêu: thấy Stream phát ra sự kiện data, end, error.

```
// file: 01-read-chunks.js
const fs = require('fs');

const s = fs.createReadStream('input.txt', {
  encoding: 'utf8',
  highWaterMark: 8
});

s.on('data', chunk => console.log('chunk:', chunk));
s.on('end', () => console.log('END'));
s.on('error', err => console.error(err.message));
```

```
# tạo dữ liệu mẫu
printf "Hello Node Stream" >
input.txt

# chạy ví dụ
node 01-read-chunks.js
```

```
Kết quả kiểm tra:
chunk: Hello No
chunk: de Strea
chunk: m
END
```

highWaterMark nhỏ giúp sinh viên dễ quan sát dữ liệu bị chia thành nhiều chunk.

Ví dụ 2: Sao chép file bằng pipe()

```
// file: 02-copy-file.js
const fs = require('fs');

const input = fs.createReadStream('input.txt');
const output = fs.createWriteStream('output.txt');

input.pipe(output);

output.on('finish', () => {
  console.log('Copy finished');
});
```

1. Chạy: node 02-copy-file.js
2. Mở output.txt.
3. So sánh nội dung output.txt với input.txt.
4. Kết luận: pipe() chuyển dữ liệu từ Readable sang Writable.

```
Kết quả kiểm tra:
Copy finished
output.txt giống input.txt
```

Ví dụ 3: Kiểm tra lỗi khi dùng Stream

Trong ứng dụng mạng, luôn phải xử lý lỗi I/O.

```
// file: 03-stream-error.js
const fs = require('fs');

const input = fs.createReadStream('missing.txt');
const output = fs.createWriteStream('out.txt');

input.on('error', err => {
  console.error('Read error:', err.message);
});

output.on('error', err => {
  console.error('Write error:', err.message);
});

input.pipe(output);
```

- Cố tình đọc file không tồn tại.
- Sinh viên quan sát event error.
- Không xử lý lỗi có thể làm ứng dụng dừng bất ngờ.
- Tư duy này áp dụng khi kết nối mạng bị lỗi hoặc mất dữ liệu.

```
Kết quả kiểm tra:
Read error: ENOENT: no such
file...
```

Ví dụ 4: Dùng pipeline() để quản lý lỗi

```
// file: 04-pipeline-copy.js
const fs = require('fs');
const { pipeline } = require('stream');

pipeline(
  fs.createReadStream('input.txt'),
  fs.createWriteStream('copy.txt'),
  err => {
    if (err) console.error('Pipeline failed:', err.message);
    else console.log('Pipeline succeeded');
  }
);
```

- pipeline() nối các Stream giống pipe().
- Điểm mạnh: lỗi của cả chuỗi xử lý gom về callback cuối.
- Khuyến nghị dùng trong ví dụ thực tế có nhiều bước xử lý.

```
Kết quả kiểm tra:
Pipeline succeeded
copy.txt được tạo
```

Transform Stream: vừa đọc vừa xử lý

- Transform Stream nhận chunk đầu vào, xử lý, rồi đẩy chunk đầu ra.
- Rất phù hợp cho các tác vụ: nén, mã hóa, lọc dữ liệu, chuẩn hóa log, chuyển đổi bản tin.
- Đây là tư duy quan trọng khi xử lý dữ liệu mạng hoặc dữ liệu cảm biến.




```
callback(null, transformedChunk);
```

Ví dụ 5: Chuyển chữ thường thành chữ hoa

Chạy trực tiếp trên terminal bằng stdin → transform → stdout.

```
// file: 05-uppercase-stdin.js
const { Transform } = require('stream');

const upper = new Transform({
  transform(chunk, enc, callback) {
    const text = chunk.toString().toUpperCase();
    callback(null, text);
  }
});

process.stdin
  .pipe(upper)
  .pipe(process.stdout);
```

```
node 05-uppercase-stdin.js
hello ctu
pipe and stream
```

```
Kết quả kiểm tra:
HELLO CTU
PIPE AND STREAM
```

Dừng chương trình: Ctrl + C

Ví dụ 6: Transform nội dung file

```
// file: 06-uppercase-file.js
const fs = require('fs');
const { Transform, pipeline } = require('stream');

const upper = new Transform({
  transform(chunk, enc, cb) {
    cb(null, chunk.toString().toUpperCase());
  }
});

pipeline(
  fs.createReadStream('input.txt'),
  upper,
  fs.createWriteStream('upper.txt'),
  err => console.log(err ? err.message : 'Done')
);
```

1. Tạo input.txt có chữ thường.
2. Chạy: node 06-uppercase-file.js
3. Mở upper.txt để kiểm tra.
4. Đây là dạng pipeline xử lý dữ liệu trước khi đưa ra đích.

```
Kết quả kiểm tra:
input.txt : hello ctu
upper.txt : HELLO CTU
```

Ví dụ 7: Đếm số byte đi qua Stream

Gần với lập trình mạng: đo lượng dữ liệu đi qua hệ thống.

```
// file: 07-count-bytes.js
const fs = require('fs');

let total = 0;

const input = fs.createReadStream('input.txt');

input.on('data', chunk => {
  total += chunk.length;
});

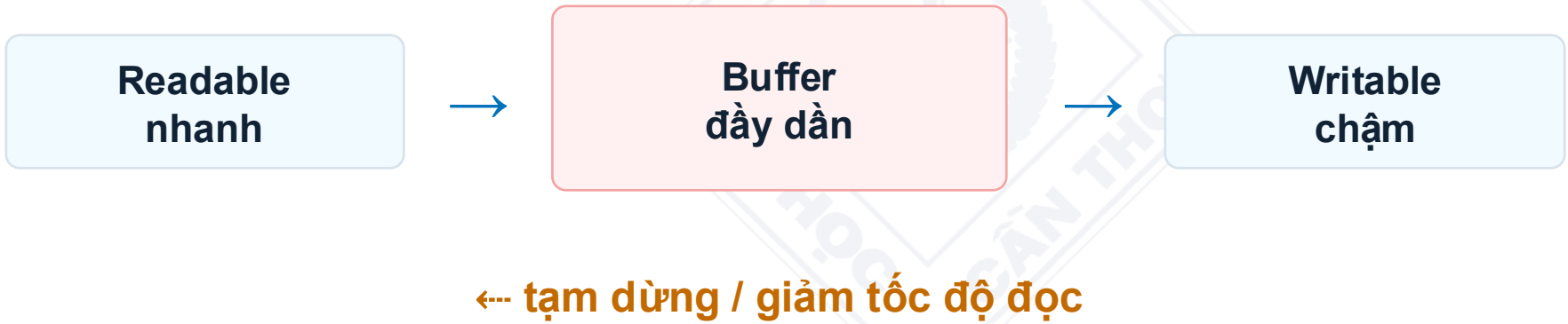
input.on('end', () => {
  console.log('Total bytes:', total);
});
```

- Mỗi chunk có độ dài riêng.
- Tổng byte cho biết kích thước dữ liệu đã đọc.
- Ứng dụng tương tự: tính dung lượng log, file upload, gói dữ liệu cảm biến.

```
Kết quả kiểm tra:
Total bytes: 17
# tùy nội dung input.txt
```

Backpressure: khi nguồn nhanh hơn đích

- Nguồn dữ liệu có thể tạo chunk rất nhanh, nhưng nơi nhận ghi/xử lý chậm hơn.
- Nếu không điều tiết, dữ liệu tích tụ trong bộ nhớ.
- pipe() và Stream API hỗ trợ cơ chế backpressure để giảm tốc độ đọc khi cần.



Ví dụ 8: Quan sát tín hiệu drain

write() trả về false khi buffer đang đầy; drain báo có thể ghi tiếp.

```
// file: 08-drain.js
const fs = require('fs');
const out = fs.createWriteStream('big.txt');
let i = 0;

function write() {
  let ok = true;
  while (i < 100000 && ok) {
    ok = out.write('line ' + i++ + '\n');
  }
  if (i < 100000) out.once('drain', write);
  else out.end(() => console.log('Done:', i));
}

write();
```

- Chạy: node 08-drain.js
- File big.txt được tạo.
- Khi buffer đầy, chương trình đợi sự kiện drain rồi ghi tiếp.
- Đây là cách điều tiết tốc độ ghi dữ liệu.

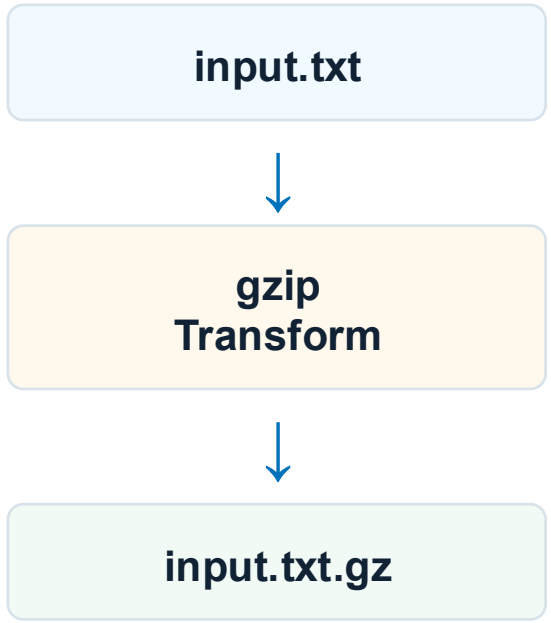
```
Kết quả kiểm tra:
Done: 100000
big.txt được tạo
```

Ví dụ 9: Nén file bằng pipeline()

Ví dụ này cho thấy chuỗi Stream: đọc file → nén → ghi file.

```
// file: 09-gzip.js
const fs = require('fs');
const zlib = require('zlib');
const { pipeline } = require('stream');

pipeline(
  fs.createReadStream('input.txt'),
  zlib.createGzip(),
  fs.createWriteStream('input.txt.gz'),
  err => console.log(err ? err.message : 'Gzip done')
);
```




```
Kết quả kiểm tra:
Gzip done
```

Ví dụ 10: Dữ liệu cảm biến dạng từng dòng

Dùng file để mô phỏng luồng dữ liệu mạng: mỗi dòng là một bản tin JSON.

```
// file: sensor.ndjson
{"id":"D01","temp":28.5}
{"id":"D02","temp":30.1}
{"id":"D01","temp":29.0}
```

- NDJSON: mỗi dòng là một JSON object.
- Rất giống luồng log hoặc bản tin cảm biến.
- Mục tiêu: đọc từng dòng, parse JSON, lọc dữ liệu cần quan tâm.
- Chưa cần client/server; chỉ luyện cách xử lý dòng dữ liệu.

Mô hình xử lý

file / nguồn dữ liệu → đọc từng dòng → parse JSON → lọc dữ liệu

Ví dụ 10: Parse JSON theo từng dòng

```
// file: 10-parse-sensor.js
const fs = require('fs');
const readline = require('readline');

const rl = readline.createInterface({
  input: fs.createReadStream('sensor.ndjson')
});

rl.on('line', line => {
  const msg = JSON.parse(line);
  if (msg.id === 'D01') {
    console.log('D01 temp:', msg.temp);
  }
});

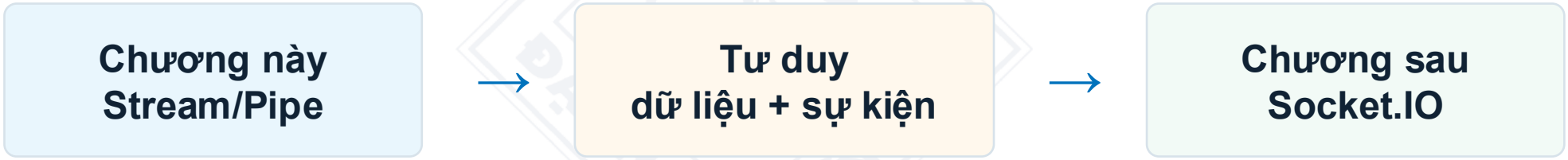
rl.on('close', () => console.log('Done'));
```

1. Tạo sensor.ndjson như slide trước.
2. Chạy: node 10-parse-sensor.js
3. Chỉ nhiệt độ của D01 được in ra.
4. Cách làm này gần với xử lý message trong ứng dụng mạng.

```
Kết quả kiểm tra:
D01 temp: 28.5
D01 temp: 29
Done
```

Hướng ứng dụng vào lập trình mạng

- Dữ liệu mạng thường không đến “một cục” mà đến theo gói, theo dòng hoặc theo message.
- Stream giúp luyện cách xử lý dữ liệu tuần tự: đọc → kiểm tra → biến đổi → ghi/gửi tiếp.
- Ở chương Socket.IO, ta sẽ áp dụng tư duy này vào sự kiện: connect, message, disconnect, data update.



Bài tập

- BT1: Đổi highWaterMark trong ví dụ đọc chunk từ 8 thành 4, 16; so sánh output.
- BT2: Thay Transform uppercase thành loại bỏ dòng trống trong file văn bản.
- BT3: Sửa ví dụ sensor.ndjson để chỉ in nhiệt độ lớn hơn 29°C.
- BT4: Tạo một file lớn và dùng count-bytes.js để kiểm tra tổng số byte.
- BT5: Cố tình đọc file không tồn tại và giải thích error event.

Tổng kết

- Stream xử lý dữ liệu theo từng chunk; phù hợp với file lớn và I/O bất đồng bộ.
- pipe() nối nguồn đọc với đích ghi; Transform Stream cho phép xử lý dữ liệu ở giữa.
- pipeline() giúp quản lý chuỗi Stream và lỗi tốt hơn.
- Backpressure giúp tránh tích tụ dữ liệu quá nhiều trong bộ nhớ.
- Các ví dụ file/log/cảm biến là bước đệm trước khi học client/server bằng Socket.IO.

CÁM ƠN

Thank you